APM52009 – Machine Learning for Scientific Computing and Numerical Analysis

Week 1: Introduction & Scope of Scientific Machine Learning

Victorita Dolean and Hadrien Montanelli

Abstract

These notes introduce the *scope* of Scientific Machine Learning (SciML) and establish a common terminology. SciML is positioned at the interface between classical numerical analysis and modern machine learning. We introduce the supervised learning problem and discuss three classical approaches to solve it without neural networks: linear regression, nonlinear regression, and trigonometric interpolation, highlighting the advantages and limitations of each. We conclude with case studies drawn from turbulence, imaging, and materials science, illustrating the application of these methods in scientific contexts. Topics related to optimization are intentionally deferred to a later chapter.

Contents

1 Scientific Computing & Numerical Analysis: The classical backbone **2**

2 What is SciML? Why now? **2**

3 The supervised learning problem **4**

3.1 Linear regression 5

3.2 Nonlinear regression 7

3.3 Trigonometric interpolation 9

4 Illustrative case studies **12**

4.1 Case study 1: Turbulence and learned closures 12

4.2 Case study 2: Computational imaging and learned inversion 12

4.3 Case study 3: Material discovery and design 13

5 Exercises **14**

6 Suggested reading **15**

1 Scientific Computing & Numerical Analysis: The classical backbone

Scientific Computing and Numerical Analysis design and analyze *numerical* algorithms for *continuous* mathematics (ODEs/PDEs, variational problems). Here is the high-level picture:

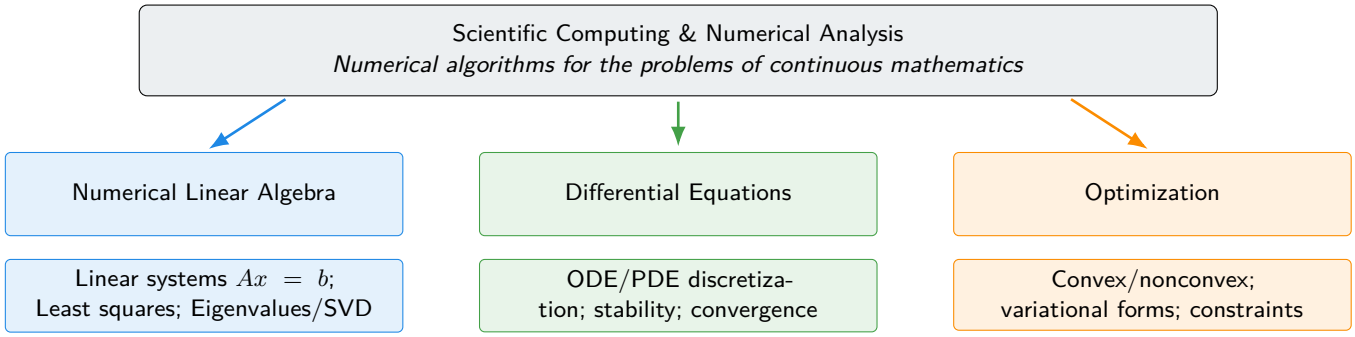

Figure 1: Classical backbone—numerical linear algebra, differential equations, and optimization.

Linear PDEs reduce to linear algebra after discretization; nonlinear PDEs are often solved by *linearize + iterate*. Problems with energies can be posed as optimization.

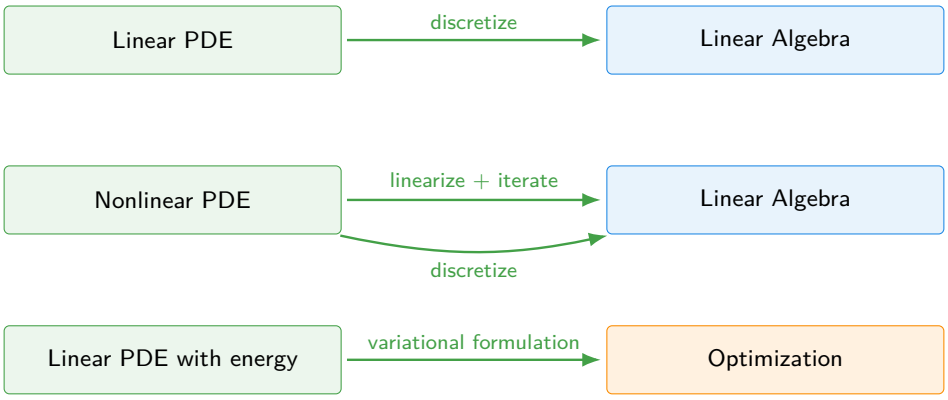

Figure 2: How PDEs connect to linear algebra and optimization.

The classical backbone of Scientific Computing and Numerical Analysis reduces complex continuous problems to algebraic and optimization structures, providing provably stable and convergent algorithms. Yet, in many modern applications, purely mechanistic (e.g., based on physical laws) modeling is limited:

- **Models may be only approximate:** relying on closure relations, empirical coefficients, or simplified constitutive laws, which can fail to capture complex real-world behavior.
- **Data are increasingly abundant:** high-dimensional and informative, offering opportunities to refine, calibrate, or even partially replace traditional models.
- **Computational demands are growing:** simulations of realistic systems often require massive resources, motivating the development of surrogate models and other accelerators.

These limitations have spurred the rise of **Scientific Machine Learning (SciML)**,[1] which augments classical numerical pipelines with data-driven components, blending mechanistic knowledge with statistical and machine learning methods while retaining interpretability, structure, and convergence guarantees.

2 What is SciML? Why now?

Working view SciML studies algorithms that integrate data, mechanistic models (e.g., ODEs, PDEs, or variational principles), and computation to solve scientific and engineering problems. The emphasis is on *structure-exploiting learning*: using known physics, symmetries, and numerical methods to guide models, improve sample efficiency, and enhance reliability.

[1] This term was introduced for the first time in this US Department of Energy (DOE) report.

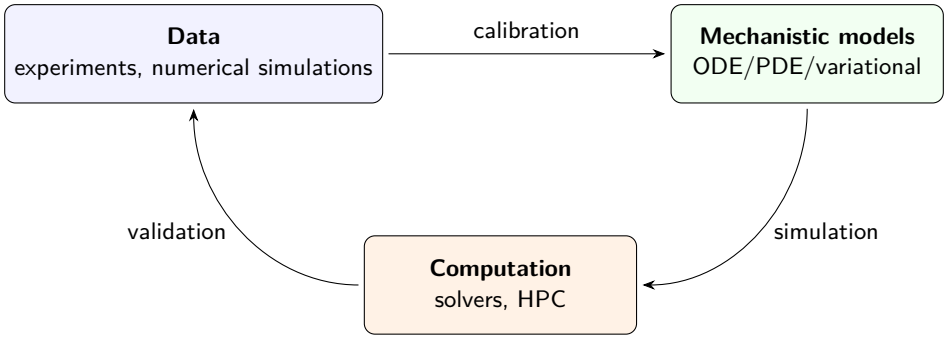

Figure 3: SciML triad—interactions between data, mechanistic models, and computation.

Scope at a glance Here are some of the important fields of SciML:

- **Model discovery:** identify governing equations or closure relations from data, uncovering underlying laws and enabling predictive modeling; approaches include sparse or symbolic regression.
- **Physics-informed and structure-preserving training:** combine supervision with PDE residuals and boundary conditions; enforce symmetries, conservation laws, and priors in strong/weak forms.
- **Operator learning and surrogates:** build fast emulators for design, uncertainty quantification, and control; techniques include reduced bases, neural surrogates, and neural operators.
- **Solver-aware learning:** unrolled or learned iterative methods, learned preconditioners, and architectures that mirror numerical schemes.
- **Hybrid mechanistic-learned models:** universal differential equations and neural closures that couple known terms with learnable components.

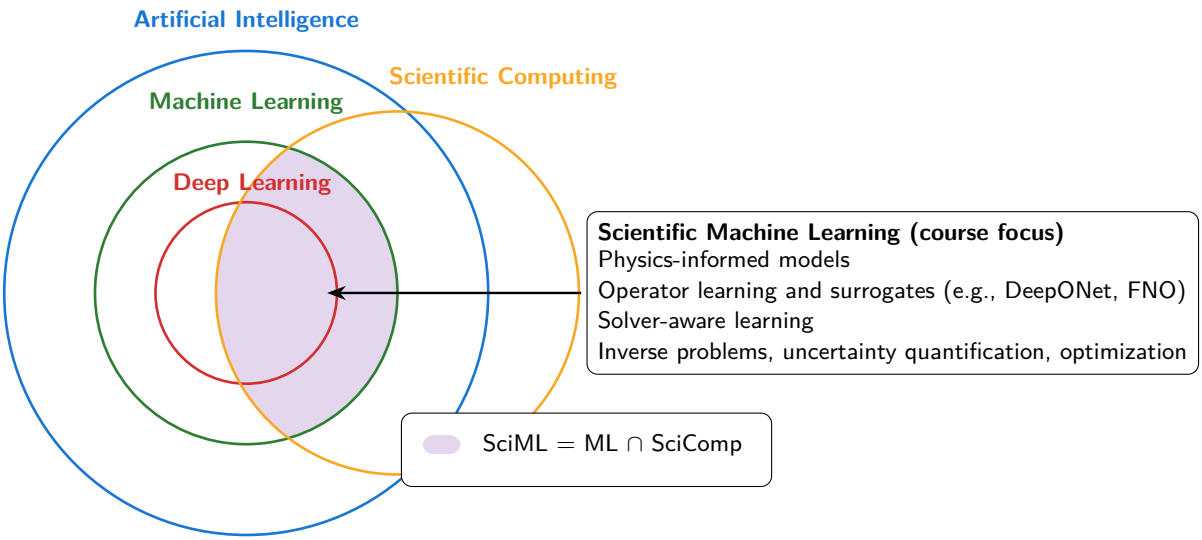

Figure 4: Nested dependencies—SciML is where Machine Learning intersects Scientific Computing.

Why now? Machine learning is transforming mathematics, physics, and engineering—from function approximation and ODE/PDE solvers to inverse problems and dynamical-system identification. Mathematics provides tools to *open the black box*—enabling principled, efficient methods with stability and convergence guarantees. Modern hardware and automatic differentiation make solver-in-the-loop learning feasible, while rising computational costs stress the value of structure-aware modeling.

Scope and boundaries SciML is not limited to neural networks; kernels, splines, polynomials, Gaussian processes, variational formulations, and classical solvers all appear, often combined. The unifying theme is *structure-exploiting learning* for scientific objectives, drawing on numerical analysis, approximation theory, functional analysis, physics, scientific computing, and data science.

3 The supervised learning problem

In Section 2, we characterized SciML as the integration of data, mechanistic models, and computation. We now formalize the *supervised learning problem*, the starting point of SciML.

Setup Let $d \geq 1$ and $D \subset \mathbb{R}^d$ be a compact domain. We are given a target function $f : D \to \mathbb{R}$ that is square-integrable, i.e., $f \in L^2(D)$.[2] From f, we observe $n \geq d$ samples

$$\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n, \qquad \boldsymbol{x}_i \in D \subset \mathbb{R}^d, \qquad y_i = f(\boldsymbol{x}_i) \in \mathbb{R}.$$

We approximate f by a parametric model $f_\theta \in F_\Theta \subset L^2(D)$, where F_Θ is a finite-dimensional space (e.g., polynomials, splines, neural networks) and $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p$, for some $p \geq 1$, are learnable parameters.

Regression vs. classification If the outputs $f(D)$ are continuous, the task is *regression*; if they take discrete values, e.g., $\{-1, 1\}$, it is *classification*. In this course, we focus on regression, since f typically represents physical quantities such as pressure or temperature.

Errors The *generalization error* measures how well f_θ approximates f over the whole domain:

$$\mathcal{E}_G(\boldsymbol{\theta}) = \|f_\theta - f\|^2_{L^2(D)} = \frac{1}{|D|} \int_D |f_\theta(\boldsymbol{x}) - f(\boldsymbol{x})|^2 \, d\boldsymbol{x}.$$

The best approximation in F_Θ corresponds to parameters

$$\boldsymbol{\theta}^*_G = \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_G(\boldsymbol{\theta}).$$

Since f is unknown on D, we replace the integral by the *training error* at the samples:

$$\mathcal{E}_T(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n |f_\theta(\boldsymbol{x}_i) - y_i|^2, \qquad \boldsymbol{\theta}^*_T = \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_T(\boldsymbol{\theta}).$$

An optimization algorithm (often gradient-based) produces an approximate minimizer $\boldsymbol{\theta}$.

Theorem 3.1 (Error decomposition)**.** *For any parameter $\boldsymbol{\theta} \in \Theta$,*

$$\mathcal{E}_G(\boldsymbol{\theta}) \leq \underbrace{\mathcal{E}_G(\boldsymbol{\theta}^*_G)}_{\textit{approximation error}} + \underbrace{2 \sup_{\boldsymbol{\theta}' \in \Theta} |\mathcal{E}_G(\boldsymbol{\theta}') - \mathcal{E}_T(\boldsymbol{\theta}')|}_{\textit{generalization gap}} + \underbrace{\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_T)}_{\textit{optimization error}}.$$

Proof

Insert and subtract intermediate terms:

$$\mathcal{E}_G(\boldsymbol{\theta}) = (\mathcal{E}_G(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta})) + (\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_G)) + (\mathcal{E}_T(\boldsymbol{\theta}^*_G) - \mathcal{E}_G(\boldsymbol{\theta}^*_G)) + \mathcal{E}_G(\boldsymbol{\theta}^*_G).$$

Then note that

$$\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_G) \leq \mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_T),$$

and both differences $\mathcal{E}_G(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta})$ and $\mathcal{E}_T(\boldsymbol{\theta}^*_G) - \mathcal{E}_G(\boldsymbol{\theta}^*_G)$ are bounded by the supremum term.

Interpretation This decomposition highlights three sources of error:

- **Approximation error:** the best the model class F_Θ can achieve (limited expressivity).
- **Generalization gap:** the mismatch between training performance/performance over the whole domain.
- **Optimization error:** how close the algorithm gets to the best possible minimizer of $\mathcal{E}_T$.

Practical note The generalization gap is typically estimated by the *test error* on unseen data.

[2] More generally, one can consider vector-valued functions $f \in L^q(D, \mathbb{R}^{d'})$ with $1 \leq q \leq \infty$ and $d' \geq 1$.

3.1 Linear regression

In the rest of this section, we focus on supervised learning methods that *do not* use neural networks and are therefore outside machine learning. They remain essential baselines in SciML for interpretability, numerical stability, and as components inside hybrid pipelines.

Model We seek a linear predictor $f_\theta : \mathbb{R}^d \to \mathbb{R}$ of the form

$$f_\theta(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x}, \qquad \boldsymbol{\theta} \in \mathbb{R}^d, \tag{3.1}$$

possibly with a bias b, which can be absorbed into $\boldsymbol{\theta}$ by augmenting each input $\boldsymbol{x}$ with a trailing 1.

Linear least squares Given $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, we define the design matrix, fitted values, and observation vector

$$X = \begin{bmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n\times d}, \qquad \boldsymbol{y}_\theta = X\boldsymbol{\theta} \in \mathbb{R}^n, \qquad \boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

The empirical risk (scaled training error) is

$$\mathcal{E}_T(\boldsymbol{\theta}) = \tfrac{1}{2}\|\boldsymbol{y}_\theta - \boldsymbol{y}\|_2^2 = \tfrac{1}{2}\|X\boldsymbol{\theta} - \boldsymbol{y}\|_2^2. \tag{3.2}$$

Problem 3.2 (Linear least squares)**.** Given $X \in \mathbb{R}^{n\times d}$ and $\boldsymbol{y} \in \mathbb{R}^n$, find

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \tfrac{1}{2}\|X\boldsymbol{\theta} - \boldsymbol{y}\|_2^2.$$

Remark 3.3 (Linearity in parameters)**.** With a feature map $\boldsymbol{\phi}(\boldsymbol{x})$, linear regression can fit nonlinear functions $f_\theta(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(\boldsymbol{x})$, e.g., polynomials. But the number of parameters grows combinatorially with degree p and dimension d, quickly becoming impractical.

Normal equations At the minimizer, the residual $\boldsymbol{r} = X\boldsymbol{\theta} - \boldsymbol{y}$ is orthogonal to $\mathrm{range}(X)$. Equivalently,

$$X^\top(X\boldsymbol{\theta} - \boldsymbol{y}) = 0 \qquad \Longleftrightarrow \qquad X^\top X\boldsymbol{\theta} = X^\top \boldsymbol{y} \qquad \text{(normal equations)}. \tag{3.3}$$

Thus the fitted values $\boldsymbol{y}_\theta = X\boldsymbol{\theta}$ are the orthogonal projection of $\boldsymbol{y}$ onto $\mathrm{range}(X)$. The projection operator is

$$H = X(X^\top X)^{-1}X^\top, \qquad \boldsymbol{y}_\theta = H\boldsymbol{y}, \quad \boldsymbol{r} = (H - I)\boldsymbol{y}.$$

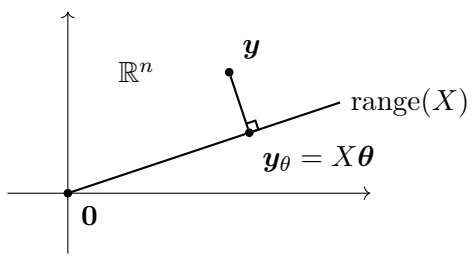

Figure 5: Geometric picture—$X\boldsymbol{\theta}$ is the orthogonal projection of $\boldsymbol{y}$ onto $\mathrm{range}(X)$.

Another way to obtain the normal equations is to compute $\nabla\mathcal{E}_T(\boldsymbol{\theta}) = X^\top(X\boldsymbol{\theta} - \boldsymbol{y})$ and set it to zero.

Theorem 3.4 (Uniqueness)**.** *The least–squares solution is unique if and only if $X^\top X$ is nonsingular, i.e., X has full column rank d.*

Numerical solution Directly forming and inverting $X^\top X$ is unstable. Robust alternatives include:

- **QR factorization:** $X = QR$ with orthogonal $Q \in \mathbb{R}^{n\times d}$ and upper-triangular $R \in \mathbb{R}^{d\times d}$. Then solve $R\boldsymbol{\theta} = Q^\top \boldsymbol{y}$ via back substitution.

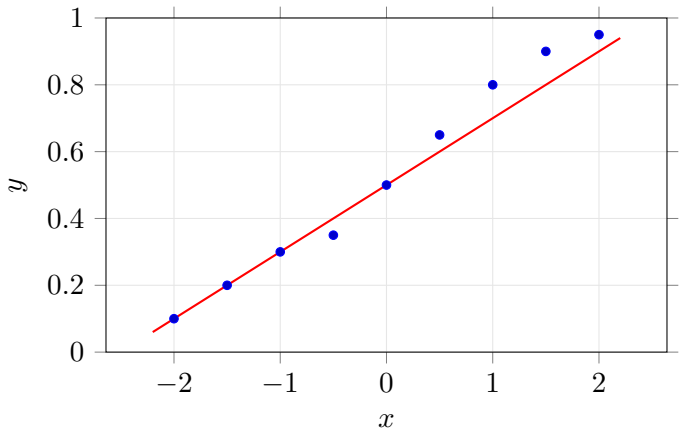

Figure 6: Example of linear regression—fitting a straight line to the data.

- **SVD:** $X = U\Sigma V^\top$ with $U \in \mathbb{R}^{n\times d}$, $V \in \mathbb{R}^{d\times d}$ orthogonal and diagonal $\Sigma \in \mathbb{R}^{d\times d}$. The solution $\boldsymbol{\theta} = V\Sigma^+U^\top\boldsymbol{y}$ uses the *Moore–Penrose pseudoinverse* Σ^+, which inverts nonzero singular values and leaves zeros, yielding the minimum-norm solution, robust to ill-conditioning or rank deficiency.

Each $X \in \mathbb{R}^{n\times d}$ of full rank has a unique QR factorization with $r_{jj} > 0$ for all $1 \le j \le d$ [12, Thm. 7.2], while each $X \in \mathbb{R}^{n\times d}$, even if rank-deficient, admits an (in general nonunique) SVD. The factorizations presented here are both in their *reduced* form. The *full* QR and SVD would produce matrices $Q \in \mathbb{R}^{n\times n}$, $R \in \mathbb{R}^{n\times d}$, $U \in \mathbb{R}^{n\times n}$, $\Sigma \in \mathbb{R}^{n\times d}$, and $V \in \mathbb{R}^{d\times d}$.

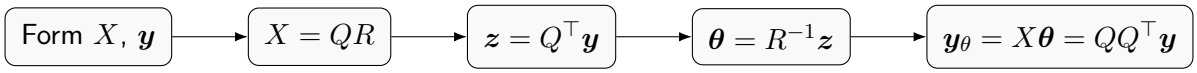

Figure 7: QR pipeline for linear least squares.

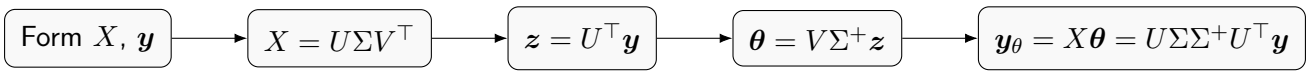

Figure 8: SVD pipeline for linear least squares.

Computational cost For $n \gg d$, both QR and SVD cost $\mathcal{O}(nd^2)$; SVD has a larger constant but handles rank-deficiency robustly. Rank-revealing QR provides a good compromise between speed and stability.

Remark 3.5 (Variants)**.** Several extensions adapt linear regression to practical needs:

- **Weighted least squares:** different samples carry different weights, minimizing $\frac{1}{2}\|W^{1/2}(X\boldsymbol{\theta}-\boldsymbol{y})\|_2^2$ with symmetric positive definite $W \in \mathbb{R}^{n\times n}$.
- **Ridge regression:** add $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ to control ill-conditioning or overfitting.
- **Lasso:** add $\lambda\|\boldsymbol{\theta}\|_1$ to promote sparsity and perform feature selection.

Takeaways

Linear regression provides a clear illustration of the three error sources from Theorem 1:

- **Approximation error:** If f is not linear, $F_\Theta = \{\text{linear functions}\}$ cannot capture it exactly.
- **Generalization gap:** Even if a linear model fits the training data well, its error on unseen samples may differ.
- **Optimization error:** Solving the normal equations eliminates this error; in practice, numerical methods (QR, SVD) achieve machine-precision minimization.

Thus linear regression is both a *baseline model* and a *pedagogical example* connecting the abstract error decomposition to concrete algorithms.

3.2 Nonlinear regression

Linear regression is powerful, but many input-output relationships are inherently nonlinear. For example, in classification, a logistic curve separates classes more effectively than a straight line.

Model In nonlinear regression, we seek a nonlinear predictor $f_\theta : \mathbb{R}^d \to \mathbb{R}$:

$$f_\theta(\boldsymbol{x}) \text{ nonlinear in } \boldsymbol{x}, \qquad \boldsymbol{\theta} \in \mathbb{R}^d, \tag{3.4}$$

with parameters $\boldsymbol{\theta}$ fitted from data. A classical case is *logistic regression*:

$$f_\theta(\boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \boldsymbol{x}}}. \tag{3.5}$$

Nonlinear least squares Given data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, define

$$\boldsymbol{y}_\theta = \begin{bmatrix} f_\theta(\boldsymbol{x}_1) \\ \vdots \\ f_\theta(\boldsymbol{x}_n) \end{bmatrix} \in \mathbb{R}^n, \qquad \boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n. \tag{3.6}$$

The training error is

$$\mathcal{E}_T(\boldsymbol{\theta}) = \tfrac{1}{2}\|\boldsymbol{y}_\theta - \boldsymbol{y}\|_2^2. \tag{3.7}$$

Problem 3.6 (Nonlinear least squares)**.** Given $\{\boldsymbol{x}_i\}_{i=1}^n \subset \mathbb{R}^d$, $\boldsymbol{y} \in \mathbb{R}^n$, and a model f_θ, find

$$\boldsymbol{\theta}^* \in \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \tfrac{1}{2}\|\boldsymbol{y}_\theta - \boldsymbol{y}\|_2^2.$$

Normal equations The residual is $\boldsymbol{r}(\boldsymbol{\theta}) = \boldsymbol{y}_\theta - \boldsymbol{y}$. Its Jacobian is

$$J(\boldsymbol{\theta}) = \frac{\partial \boldsymbol{y}_\theta}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{n \times d}, \qquad J_{ij}(\boldsymbol{\theta}) = \frac{\partial f_\theta(\boldsymbol{x}_i)}{\partial \theta_j}. \tag{3.8}$$

Then $\nabla \mathcal{E}_T(\boldsymbol{\theta}) = J(\boldsymbol{\theta})^\top \boldsymbol{r}(\boldsymbol{\theta})$. Stationarity requires solving $J(\boldsymbol{\theta})^\top \boldsymbol{r}(\boldsymbol{\theta}) = 0$, a nonlinear system in $\boldsymbol{\theta}$:

$$J(\boldsymbol{\theta})^\top \boldsymbol{r}(\boldsymbol{\theta}) = 0 \qquad \text{(nonlinear normal equations)}. \tag{3.9}$$

Gauss–Newton method Since the normal equations are nonlinear, we solve them iteratively. Gauss–Newton linearizes the model around the current iterate $\boldsymbol{\theta}^k$:

$$\boldsymbol{y}_{\boldsymbol{\theta}^{k+1}} \approx \boldsymbol{y}_{\boldsymbol{\theta}^k} + J(\boldsymbol{\theta}^k)(\boldsymbol{\theta}^{k+1} - \boldsymbol{\theta}^k). \tag{3.10}$$

The resulting update step is

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k + \boldsymbol{s}^k, \qquad \text{where } \boldsymbol{s}^k \text{ satisfies } J(\boldsymbol{\theta}^k)^\top J(\boldsymbol{\theta}^k)\, \boldsymbol{s}^k = -J(\boldsymbol{\theta}^k)^\top \boldsymbol{r}(\boldsymbol{\theta}^k). \tag{3.11}$$

For convergence of the Gauss–Newton method, we refer to [8, Thm. 10.1].

Algorithm 1 Gauss–Newton with QR Factorization

Require: Nonlinear least-squares residual $\boldsymbol{r}(\boldsymbol{\theta}) = \boldsymbol{y}_\theta - \boldsymbol{y}$, Jacobian $J(\boldsymbol{\theta})$, tolerance tol > 0

```
Choose initial guess θ^0 ∈ R^d; set k ← 0
repeat
    Evaluate Jacobian and residual: J^k ← J(θ^k), r^k ← r(θ^k)
    Compute (reduced) QR factorization: J^k = Q^k R^k
    Solve the upper-triangular system: R^k s^k = −(Q^k)^⊤ r^k
    Update parameters: θ^{k+1} ← θ^k + s^k
    Evaluate new residual: r^{k+1} ← r(θ^{k+1})
    k ← k + 1
until ‖s^{k−1}‖_2 ≤ tol or ‖r^k‖_2 ≤ tol
return θ^k
```

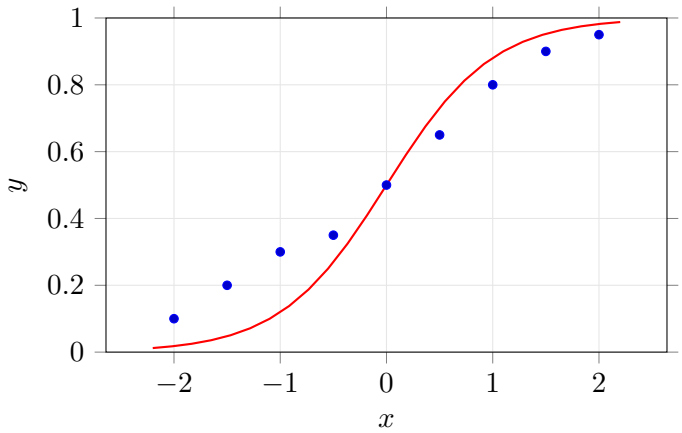

Figure 9: Example of nonlinear regression—fitting a logistic curve to the data.

Computational cost Each iteration costs $\mathcal{O}(nd^2)$ for forming J^k and solving the least-squares step via QR. SVD is more robust for ill-conditioned J^k, but more expensive. Rank-revealing QR is, again, a good compromise.

Remark 3.7 (Beyond Gauss–Newton)**.** Several enhancements are commonly used in practice:

- **Levenberg–Marquardt (LM):** adds damping, interpolating between Gauss–Newton and gradient descent; robust when $J^\top J$ is nearly singular.
- **Quasi–Newton (e.g., BFGS):** avoid explicit Jacobians, useful in high dimensions.
- **Stochastic/online variants:** approximate $J^\top J$ using mini-batches for large-scale problems.

Takeaways

Compared with linear regression:

- **Approximation error:** a nonlinear function class $\hat{F}$ can better capture nonlinear f.
- **Generalization gap:** often larger, since nonlinear models are more prone to overfitting; cross-validation and regularization help control it.
- **Optimization error:** no closed form exists; iterative methods (Gauss–Newton, LM, BFGS) are required, and convergence depends on initialization and conditioning.

Nonlinear regression illustrates the trade-off between *expressivity* and *optimization complexity*.

Table 1: Linear vs. nonlinear regression—differ sharply in expressivity, optimization, and error behavior.

	Linear regression	**Nonlinear regression**
Model class	Affine: $f_\theta(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x}$	Nonlinear: $f_\theta(\boldsymbol{x})$ flexible
Error landscape	Convex; unique minimum	Nonconvex; multiple minima
Optimization	Closed form $(X^\top X)^{-1}X^\top \boldsymbol{y}$; via QR/SVD for stability	Iterative; via Gauss–Newton, LM, quasi–Newton
Computation	Single solve; $\mathcal{O}(nd^2)$	Many solves; $\mathcal{O}(nd^2)$ each
Approximation error	Large if f nonlinear (bias)	Smaller; can capture complex f
Generalization gap	Small (low variance)	Larger; requires regularization
Optimization error	Zero (exact projection)	Nonzero; depends on init. & cond.

3.3 Trigonometric interpolation

Linear and nonlinear regression can approximate general functions, but for *periodic* data, trigonometric polynomials are the natural choice, forming the basis of Fourier analysis and spectral methods.

Model On the interval $[0, 2\pi]$, understood periodically (0 and 2π are identified [6, Chap. 1]), we seek a trigonometric polynomial $f_\theta : [0, 2\pi] \to \mathbb{C}$ (here, f can be complex-valued)

$$f_\theta(x) = \sum_{k=-(n-1)/2}^{(n-1)/2} \theta_k e^{ikx}, \qquad \text{(trigonometric interpolation)}, \tag{3.12}$$

where n is assumed odd. The coefficients $\boldsymbol{\theta} = \{\theta_k\}$ are chosen so that f_θ interpolates given samples.

Trigonometric interpolation Let equispaced nodes be

$$x_i = \frac{2\pi i}{n}, \qquad 0 \le i \le n-1,$$

with data $\{(x_i, y_i)\}_{i=0}^{n-1}$. (We use 0-based indexing, more common in Fourier analysis.) We require

$$\boldsymbol{y}_\theta = \boldsymbol{y}, \qquad \boldsymbol{y}_\theta = \begin{bmatrix} f_\theta(x_0) \\ \vdots \\ f_\theta(x_{n-1}) \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} y_0 \\ \vdots \\ y_{n-1} \end{bmatrix}.$$

Problem 3.8 (Trigonometric interpolation)**.** Given equispaced points $\{x_i\}_{i=0}^{n-1} \in [0, 2\pi]$ and values $\boldsymbol{y} \in \mathbb{C}^n$, find $\boldsymbol{\theta} \in \mathbb{C}^n$ such that $\boldsymbol{y}_\theta = \boldsymbol{y}$.

Fourier series connection For $f \in L^2([0, 2\pi])$, the partial sums

$$f_m(x) = \sum_{k=-m}^{m} c_k e^{ikx}, \qquad c_k = \frac{1}{2\pi} \int_0^{2\pi} f(x) e^{-ikx}\, dx \tag{3.13}$$

converge to f in the $L^2([0, 2\pi])$-norm [6, Thm. 1.11, Chap. 3], and we write

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ikx}. \tag{3.14}$$

The partial sum in Eq. (3.13) is a *trigonometric polynomial of degree* m, the c_k are the *Fourier coefficients* of f, and Eq. (3.14) is its *Fourier series*. For real f, $c_{-k} = \bar{c}_k$. Continuity alone does not ensure pointwise convergence: some continuous functions have Fourier series that diverge at points [15, Thm. 1.1, Chap. 8], whereas Lipschitz functions have a unique absolutely convergent Fourier series [15, Thm. 3.6, Chap. 6].

Projection vs. interpolation The *degree-m projection* in Eq. (3.13) is the least-squares best approximation using Fourier coefficients c_k, but requires access to $f(x)$ for all x. The *degree-m interpolant* in Eq. (3.12) is simpler: it uses only sample values at x_i, with coefficients approximated via the trapezoidal rule [13]:

$$\theta_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j e^{-ikx_j} \;\approx\; c_k. \tag{3.15}$$

Fast Fourier Transform (FFT) The discrete Fourier transform (DFT) of $\boldsymbol{y} = (y_0, \ldots, y_{n-1})^\top$ is

$$\theta_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j e^{-2\pi ijk/n}, \qquad 0 \le k \le n-1,$$

which naively costs $\mathcal{O}(n^2)$. When n is even, the Fast Fourier Transform (FFT) splits $\boldsymbol{y}$ into even and odd:

$$\theta_k = \frac{1}{n} \sum_{j=0}^{n/2-1} y_{2j} e^{-2\pi i(2j)k/n} + \frac{e^{-2\pi ik/n}}{n} \sum_{j=0}^{n/2-1} y_{2j+1} e^{-2\pi i(2j)k/n}.$$

These two sums are DFTs of length $n/2$, reused for both k and $k+n/2$, giving the recurrence

$$T(n) = 2T(n/2) + \mathcal{O}(n),$$

whose solution is $T(n) = \mathcal{O}(n \log n)$. Modern FFT codes handle both even and odd n efficiently.

Computational cost The FFT therefore costs $\mathcal{O}(n \log n)$.

Algorithm 2 Cooley–Tukey FFT (radix-2, recursive)

Require: Vector $\boldsymbol{y} = (y_0, \ldots, y_{n-1}) \in \mathbb{C}^n$ with $n = 2^m$
Ensure: $\boldsymbol{\theta} = \mathrm{DFT}_n(\boldsymbol{y})$, i.e., $\theta_k = \frac{1}{n}\sum_{j=0}^{n-1} y_j e^{-2\pi i jk/n}$

```
if n = 1 then
    return y
end if
Split y into even/odd parts:
    y^even = (y_0, y_2, ..., y_{n-2}),   y^odd = (y_1, y_3, ..., y_{n-1})
θ^even ← FFT(y^even)                          ▷ DFT of length n/2
θ^odd ← FFT(y^odd)                            ▷ DFT of length n/2
Let z = e^{-2πi/n}                            ▷ Primitive nth root of unity
for k = 0 to n/2 − 1 do
    t ← z^k θ_k^odd
    θ_k ← θ_k^even + t
    θ_{k+n/2} ← θ_k^even − t
end for
return θ
```

Remark 3.9 (Even n)**.** For $n = 2m$, uniqueness is ensured by imposing $\theta_{-m} = \theta_m$ and writing

$$f_\theta(x) = \sum_{k=-m}^{m}{}' \; \theta_k e^{ikx},$$

where the prime indicates that the terms with $k = \pm m$ are halved. FFT codes typically store

$$\boldsymbol{\theta} = \big((\theta_{-m} + \theta_m)/2,\, \theta_{-m+1}, \ldots, \theta_{m-1}\big)^\top.$$

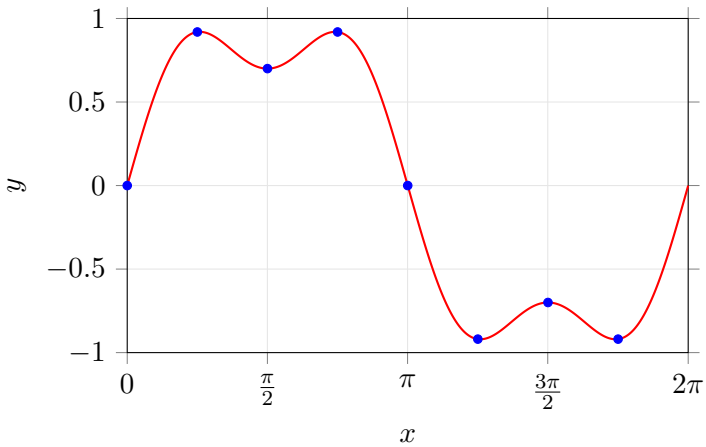

Figure 10: Trigonometric interpolation on $[0, 2\pi]$—the interpolant passes exactly through all data points.

Theorem 3.10 (Convolution)**.** *If $f, g \in L^2([0, 2\pi])$ with coefficients c_k and d_k, then*

$$h(x) = \int_{[0,2\pi]} g(x-y) f(y) dy \in L^2([0, 2\pi]) \qquad \textit{and} \qquad h(x) = \sum_{k\in\mathbb{Z}} c_k d_k e^{ikx}.$$

Definition 3.11 (Sobolev space)**.** For $s \in \mathbb{R}$, $H^s([0,2\pi])$ is the space of functions with

$$\|f\|^2_{H^s([0,2\pi])} = \sum_{k\in\mathbb{Z}} (1+|k|^2)^s |c_k|^2 < \infty.$$

Higher dimensions In dimension $d \geq 1$, the Fourier series of a function $f \in L^2([0,2\pi]^d)$ reads

$$f(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} c_{\boldsymbol{k}} e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \qquad c_{\boldsymbol{k}} = \frac{1}{(2\pi)^d} \int_{[0,2\pi]^d} f(\boldsymbol{x}) e^{-i\boldsymbol{k}\cdot\boldsymbol{x}}\, d\boldsymbol{x}.$$

We seek a trigonometric interpolant $f_\theta : [0,2\pi]^d \to \mathbb{C}$ of the form

$$f_\theta(\boldsymbol{x}) = \sum_{k_1=-(n_1-1)/2}^{(n_1-1)/2} \cdots \sum_{k_d=-(n_d-1)/2}^{(n_d-1)/2} \theta_{\boldsymbol{k}}\, e^{i\boldsymbol{k}\cdot\boldsymbol{x}},$$

with coefficients (computed via the multidimensional FFT)

$$\theta_{\boldsymbol{k}} = \frac{1}{n_1\cdots n_d} \sum_{j_1=0}^{n_1-1} \cdots \sum_{j_d=0}^{n_d-1} f(x_{j_1},\ldots,x_{j_d})\, e^{-i\boldsymbol{k}\cdot(x_{j_1},\ldots,x_{j_d})},$$

and grid points

$$(x_{j_1},\ldots,x_{j_d}) = \Big(\frac{2\pi j_1}{n_1},\ldots,\frac{2\pi j_d}{n_d}\Big), \qquad 0 \leq j_1 \leq n_1-1,\ldots,0 \leq j_d \leq n_d-1.$$

Theorem 3.12 (Convolution in $\mathbb{R}^d$)**.** *If $f, g \in L^2([0,2\pi]^d)$ with coefficients $c_{\boldsymbol{k}}$ and $d_{\boldsymbol{k}}$, then*

$$h(\boldsymbol{x}) = \int_{[0,2\pi]^d} g(\boldsymbol{x}-\boldsymbol{y}) f(\boldsymbol{y}) d\boldsymbol{y} \quad \in L^2([0,2\pi]^d) \qquad \textit{and} \qquad h(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} c_{\boldsymbol{k}} d_{\boldsymbol{k}} e^{i\boldsymbol{k}\cdot\boldsymbol{x}}.$$

Definition 3.13 (Sobolev space in $\mathbb{R}^d$)**.** For $s \in \mathbb{R}$, $H^s([0,2\pi]^d)$ is the space of functions with

$$\|f\|^2_{H^s([0,2\pi]^d)} = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} (1+\|\boldsymbol{k}\|_2^2)^s |c_{\boldsymbol{k}}|^2 < \infty.$$

Takeaways

Trigonometric interpolation is a natural choice when data are *periodic*. It offers several benefits:

- **Respects periodicity:** boundary conditions are enforced automatically.
- **Spectral accuracy:** for analytic functions, Fourier coefficients decay exponentially, yielding exponential convergence.
- **Computational efficiency:** the FFT reduces complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n\log n)$ in $\mathbb{R}$, and from $\mathcal{O}(n_1^2\cdots n_d^2)$ to $\mathcal{O}(n_1\cdots n_d \log(n_1\cdots n_d))$ in $\mathbb{R}^d$.
- **Operator diagonalization:** convolution and many PDE operators become diagonal in Fourier space, enabling highly efficient solvers.

Compared with regression methods:

- **Linear regression:** affine, convex, closed form.
- **Nonlinear regression:** expressive, nonconvex, iterative.
- **Trigonometric interpolation:** periodic, linear, FFT-based.

Lesson for SciML: choose a model that captures the data structure (linear, nonlinear, or periodic) and a solver that is numerically stable (QR, SVD, FFT) to make the method practical.

4 Illustrative case studies

To appreciate why modern scientific problems require new tools beyond classical numerical analysis, it is instructive to look at a few representative examples. In all three cases below, traditional deterministic solvers or straightforward data-driven regressions provide partial answers, but fail to fully capture the complexity, constraints, or structure of the underlying physics. SciML extends the numerical toolbox through *structure-aware learning*: combining data, physical laws, and solvers in a unified framework.

Throughout the course, we will study the main tools that make this integration possible:

- **Physics-informed networks:** encode symmetries, conservation, and boundary conditions.
- **Operator-learning architectures:** learn mappings between functions, not just scalar outputs.
- **Multi-fidelity surrogates:** fuse data across scales or accuracies.
- **Solver-unrolling:** integrate iterative numerical schemes into trainable networks.

The following case studies illustrate why these ideas are needed and will be revisited in later lectures when we formalize the corresponding algorithms.

4.1 Case study 1: Turbulence and learned closures

Context Turbulent flows obey the incompressible Navier–Stokes equations:

$$\partial_t \bar{u} + (\bar{u}\cdot\nabla)\bar{u} + \nabla\bar{p} - \nu\Delta\bar{u} = -\nabla\cdot\tau_{\mathsf{SGS}}, \quad \nabla\cdot\bar{u} = 0.$$

Direct Numerical Simulation (DNS) resolves all scales but is intractable for realistic Reynolds numbers. Large-Eddy Simulation (LES) filters out small scales and introduces a subgrid stress tensor τ_{SGS} that must be modeled.

Where basic ML helps Regression models can learn $\tau_{\mathsf{SGS}} \approx \Phi_\theta(\nabla\bar{u})$ from DNS data. This reduces empirical error and improves prediction compared with classical eddy-viscosity models.

Why go beyond Generic regressors ignore:

- **Invariance:** $\Phi_\theta(R\nabla\bar{u}R^\top) = R\Phi_\theta(\nabla\bar{u})R^\top$ for any rotation R.
- **Energy dissipation:** $\varepsilon = -\tau_{\mathsf{SGS}} : \nabla\bar{u} \geq 0$ for stability.

SciML introduces *tensor-basis networks* and *energy-constrained losses* that preserve these properties by design. We will later study these in the module on **structure-preserving learning**.

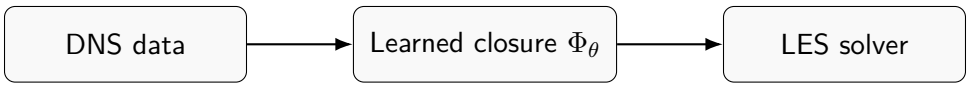

Figure 11: Data-driven closure modeling pipeline: DNS data → learned model → LES solver.

4.2 Case study 2: Computational imaging and learned inversion

Context Inverse problems such as computed tomography (CT), magnetic resonance imaging (MRI), or positron emission tomography (PET) aim to reconstruct an image $\boldsymbol{x}$ from noisy measurements $\boldsymbol{y} \approx A(\boldsymbol{x}) + \boldsymbol{\varepsilon}$. Classical approaches estimate $\boldsymbol{x}$ by solving

$$\min_{\boldsymbol{x}} \ \tfrac{1}{2}\|A(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \lambda R(\boldsymbol{x}),$$

where A is the forward operator, possibly nonlinear, and $R(\boldsymbol{x})$ is a handcrafted regularizer (e.g., Ridge or total variation penalty).

Where basic ML helps Convolutional networks such as U-Nets can learn mappings $\boldsymbol{y} \mapsto \boldsymbol{x}_\theta$ that effectively denoise and deblur data, often outperforming classical regularizers in practice.

Why go beyond Purely data-driven models may ignore the underlying physics ($A(\boldsymbol{x}_\theta) \neq \boldsymbol{y}$) or hallucinate features absent from the measurements. SciML addresses these issues through:

- **Physics-informed losses:** enforce consistency by penalizing $\|A(\boldsymbol{x}_\theta) - \boldsymbol{y}\|_2^2$.
- **Unrolled optimization:** learn iterative update rules of the form
$$\boldsymbol{x}_\theta^{k+1} = \boldsymbol{x}_\theta^k - \eta_k \nabla_x \left(\tfrac{1}{2}\|A(\boldsymbol{x}_\theta^k) - \boldsymbol{y}\|_2^2 + R_\theta(\boldsymbol{x}_\theta^k) \right),$$
combining interpretability with trainable components.

We will revisit these ideas under **solver-aware learning** and **learned regularization**.

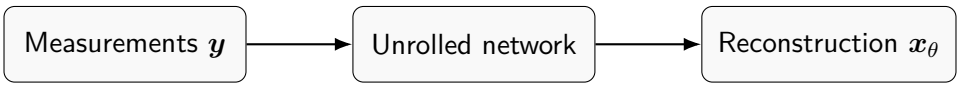

Figure 12: Learned inversion unrolls an iterative solver into a trainable network.

4.3 Case study 3: Material discovery and design

Context Discovering new materials requires predicting macroscopic properties from atomic configurations. Quantum-scale simulations (e.g., density-functional theory) are accurate but expensive.

Where basic ML helps Regression models can learn
$$\boldsymbol{y} \approx \Phi_\theta(\boldsymbol{x}), \quad \boldsymbol{x} = \text{atomic descriptor},$$
predicting formation energy or stability across databases, accelerating screening by orders of magnitude.

Why go beyond Basic ML neglects symmetries and scale coupling:

- **Invariances:** properties must be unchanged under translation, rotation, and permutation of atoms.
- **Multi-scale coupling:** electronic, atomistic, and continuum effects interact.

SciML uses *graph neural networks* for symmetry preservation and *multi-fidelity surrogates* to link quantum and experimental data. These ideas reappear in **operator learning** and **multi-scale modeling**.

Figure 13: Graph-based SciML models respect atomic symmetries and interactions.

Takeaways

These case studies illustrate that moving beyond classical numerics is not about replacing mathematical models with data, but about *integrating* them. SciML preserves the rigor of numerical analysis while extending its reach to data-rich, structure-dependent systems. In particular:

- **Turbulence:** learning closures respects physical invariances and energy dissipation, improving LES predictions.
- **Imaging:** unrolled networks combine measurement physics with data-driven priors to reduce artifacts and speed up reconstruction.
- **Materials:** graph-based models enforce atomic symmetries and multi-scale correlations, enabling accurate predictions across chemical space.

Overall, SciML unifies data, physics, and numerics into a coherent framework, allowing us to tackle problems where neither pure simulation nor pure data-driven methods suffice.

5 Exercises

Exercise 5.1. Given $X \in \mathbb{R}^{n\times d}$ with $n \geq d$, show that $X^\top X$ is nonsingular if and only if X has full column rank d.

Exercise 5.2 (Rank)**.** Let $X \in \mathbb{R}^{n\times d}$ with $n \geq d$. Show its rank using two different factorizations:

1. Let $X = QR$ be a reduced QR factorization, with $R \in \mathbb{R}^{d\times d}$ upper triangular. Show that X has rank r if and only if exactly r diagonal entries r_{ii} of R are nonzero.
2. Let $X = U\Sigma V^\top$ be a reduced singular value decomposition, with $\Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_d)$. Show that X has rank r if and only if exactly r singular values σ_i are nonzero.

Exercise 5.3 (Regularization)**.** Ridge regression regularizes by minimizing

$$\mathcal{E}_T(\boldsymbol{\theta}) = \tfrac{1}{2}\|X\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 + \tfrac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2.$$

1. Show that the normal equations read
$$X^\top(X\boldsymbol{\theta} - \boldsymbol{y}) + \lambda\boldsymbol{\theta} = 0.$$
2. When no regularization is applied, the SVD solution is
$$\boldsymbol{\theta} = V\Sigma^+U^\top\boldsymbol{y}, \qquad \Sigma^+ = \operatorname{diag}\Big(\sigma_1^{-1}, \ldots, \sigma_r^{-1}, 0, \ldots, 0\Big),$$
where $r = \operatorname{rank}(X)$. Show that with Ridge regularization it becomes
$$\boldsymbol{\theta} = V \operatorname{diag}\left(\frac{\sigma_1}{\sigma_1^2 + \lambda}, \ldots, \frac{\sigma_r}{\sigma_r^2 + \lambda}, 0, \ldots, 0\right) U^\top\boldsymbol{y}.$$
3. Show that we can rewrite the minimization problem in the augmented form
$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \tfrac{1}{2}\|\tilde{X}\boldsymbol{\theta} - \tilde{\boldsymbol{y}}\|_2^2$$
with $\tilde{X} \in \mathbb{R}^{(n+d)\times d}$ and $\tilde{\boldsymbol{y}} \in \mathbb{R}^{n+d}$.
4. With the augmented system, we can still solve via $\tilde{X} = \tilde{Q}\tilde{R}$. The cost of (non-augmented) QR with Householder reflections is $\mathcal{O}(2nd^2 - \frac{2}{3}d^3)$. What is the cost in the augmented case?

Exercise 5.4 (Gauss–Newton vs. Newton)**.** Consider a nonlinear least-squares cost

$$\mathcal{E}_T(\boldsymbol{\theta}) = \tfrac{1}{2}\sum_{i=1}^{n} r_i(\boldsymbol{\theta})^2,$$

with residual vector $\boldsymbol{r}(\boldsymbol{\theta}) = (r_1(\boldsymbol{\theta}), \ldots, r_n(\boldsymbol{\theta}))^\top$.

1. The Newton update for minimizing $\mathcal{E}_T$ reads
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - H(\boldsymbol{\theta}^k)^{-1}\nabla\mathcal{E}_T(\boldsymbol{\theta}^k),$$
Show that
$$\nabla\mathcal{E}_T(\boldsymbol{\theta}) = J(\boldsymbol{\theta})^\top\boldsymbol{r}(\boldsymbol{\theta}), \quad H(\boldsymbol{\theta}) = J(\boldsymbol{\theta})^\top J(\boldsymbol{\theta}) + \sum_{i=1}^{n} r_i(\boldsymbol{\theta})\nabla^2 r_i(\boldsymbol{\theta}),$$
where $J(\boldsymbol{\theta}) = \partial\boldsymbol{r}/\partial\boldsymbol{\theta}$ is the Jacobian and $H(\boldsymbol{\theta})$ is the Hessian of the residuals.
2. Show that the Gauss–Newton method ignores the second term in $H(\boldsymbol{\theta})$ and thus corresponds to Newton's method with the Hessian replaced by ${J^k}^\top J^k$:
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - ({J^k}^\top J^k)^{-1}{J^k}^\top\boldsymbol{r}^k, \quad J^k = J(\boldsymbol{\theta}^k), \quad \boldsymbol{r}^k = \boldsymbol{r}(\boldsymbol{\theta}^k).$$

6 Suggested reading

Some material follows [1]. For least squares, see APM 41012, APM 51435, and [8, 12]. For Fourier analysis, see [5, 6, 15]; beyond the FFT [3], related algorithms include the trapezoidal rule [13] and companion-matrix root methods [2], with practical aspects in [7, 14]. Other regression covers Chebyshev interpolation [11], tensor-product grids, and tensor-train decomposition [9]. For classification, baselines include nearest neighbors, SVMs, decision trees, and random forests; see APM 51053, CSC 51054, and [4].

Table 2: *TL;DR table of 15 favorite references. Courses, books, and key papers grouped by topic.*

	Courses @ X	Books	Key papers
Supervised learning	APM 51053 & CSC 51054	[4]	[1]
Least squares	APM 41012 & APM 51435	[8, 12]	–
Fourier theory	FMA 3X061 & FMA 42032	[5, 6, 11, 15]	–
Fourier practice	APM 51055	[11]	[2, 3, 7, 13, 14]
Chebyshev & high dimensions	APM 41012	[11]	[9, 10]

References

[1] J. Berner, P. Grohs, G. Kutyniok, and P. Petersen, *The modern mathematics of deep learning*, in Mathematical Aspects of Deep Learning, P. Grohs and G. Kutyniok, eds., Cambridge University Press, Cambridge, 2023, pp. 1–111.

[2] J. P. Boyd, *Computing the zeros, maxima and inflection points of Chebyshev, Legendre and Fourier series: Solving transcendental equations by spectral interpolation and polynomial rootfinding*, J. Eng. Math., 56 (2006), pp. 203–219.

[3] J. W. Cooley and J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Math. Comp., 19 (1965), pp. 297–301.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, Cambridge, 2016.

[5] P. Henrici, *Applied and Computational Complex Analysis*, Wiley, New York, 1986.

[6] Y. Katznelson, *An Introduction to Harmonic Analysis*, Cambridge University Press, Cambridge, 2004.

[7] H. Montanelli, *Numerical Algorithms for Differential Equations with Periodicity*, PhD thesis, University of Oxford, 2017.

[8] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, New York, 2nd ed., 2006.

[9] I. V. Oseledets, *Tensor-train decompositions*, SIAM J. Sci. Comput., 33 (2011), pp. 2295–2317.

[10] L. N. Trefethen, *Cubature, approximation, and isotropy in the hypercube*, SIAM Rev., 59 (2017), pp. 469–491.

[11] ——, *Approximation Theory and Approximation Practice*, SIAM, Philadelphia, extended ed., 2019.

[12] L. N. Trefethen and D. Bau, *Numerical Linear Algebra, Twenty-fifth Anniversary Edition*, SIAM, Philadelphia, 2022.

[13] L. N. Trefethen and J. A. C. Weideman, *The exponentially convergent trapezoidal rule*, SIAM Rev., 56 (2014), pp. 385–458.

[14] G. B. Wright, M. Javed, H. Montanelli, and L. N. Trefethen, *Extension of Chebfun to periodic functions*, SIAM J. Sci. Comput., 37 (2015), pp. C554–C573.

[15] A. Zygmund, *Trigonometric Series*, Cambridge University Press, Cambridge, 1959.